

31-Feature Engineering

October 22, 2024

1 Feature Engineering

Feature engineering is the process of transforming raw data into meaningful features that better represent the underlying problem for predictive models. In simple terms, you are crafting the inputs (features) that will help your machine learning algorithm understand patterns in the data.

Think of your data as ingredients, and feature engineering is like preparing the ingredients to make the best possible dish (your model).

1.0.1 Why is Feature Engineering Important?

Even the most powerful algorithms can't perform well if the input features are poor. Good feature engineering improves the accuracy and predictive power of machine learning models because:

- It helps capture the underlying structure of the data.
- It makes relationships between variables clearer to the model.
- It reduces noise and removes irrelevant or redundant features.

1.0.2 Key Steps in Feature Engineering

1. Understanding the Data

- Before engineering features, you must explore the data to understand its characteristics. Look at data types (numerical, categorical), distributions, missing values, etc.
- Example: If you're working with a dataset of house prices, the features could include the number of bedrooms, location, size, etc.

2. Handling Missing Values

- Missing data can reduce model performance. You need to decide whether to:
 - Remove rows with missing values.
 - Impute missing values using techniques like mean/median imputation for numerical data or mode for categorical data.

3. Encoding Categorical Variables

- Categorical features are non-numeric (e.g., "male" or "female").
 - Use One-Hot Encoding for non-ordinal categorical variables (e.g., for cities: "New York", "San Francisco").

- Use Ordinal Encoding for ordinal variables (e.g., “low”, “medium”, “high”).

4. Scaling and Normalization

- Numerical features might have different scales (e.g., age in years vs. salary in dollars). Scaling ensures all features contribute equally.
 - **Standardization:** Scales data to have zero mean and unit variance (used in algorithms like Logistic Regression, SVM).
 - **Normalization:** Scales features to be within a specific range, like [0,1].

5. Creating Interaction Features

- Sometimes the interaction between features holds valuable information. For example, in a housing dataset, combining features like “number of bedrooms” and “square footage” could create a new feature that represents the “density” of a house.

6. Polynomial Features

- In some cases, raising features to a power or creating interaction terms between them can help the model capture more complex relationships. Example: For a linear relationship like $y=aX+b$, adding as a feature allows the model to capture quadratic (curved) relationships.

7. Binning or Discretization

- Convert continuous variables into discrete categories (e.g., age into age groups: “young”, “middle-aged”, “senior”).
- This can help simplify the model or capture patterns that only emerge in specific ranges.

8. Feature Selection

- Not all features are equally useful. Feature selection involves choosing the most relevant features and discarding irrelevant or redundant ones to prevent overfitting and reduce complexity.
- Methods: Correlation matrix, Lasso regression, Recursive Feature Elimination (RFE).

1.1 Feature Engineering in Classification

Feature engineering is essential in classification because it helps separate different classes by creating better and more representative features for the learning algorithm. In other words, the right features will make the difference between a model that performs poorly and one that can classify instances correctly with high accuracy.

A good classification model will find patterns in the features that distinguish the classes from one another. The task of feature engineering is to create these distinguishing features.

1.1.1 1. Understanding the Problem and Domain Knowledge

Understanding the problem domain is crucial. The more you know about the field (whether it’s finance, healthcare, or marketing), the better you’ll be able to craft relevant features. Domain expertise helps you identify which aspects of the data could be important for classifying observations.

Example: In a customer churn prediction problem, knowing that “tenure” (how long a customer has been with the company) might impact churn can help you create a “tenure bucket” feature.

1.1.2 2. Handling Missing Values

Missing data is a common problem in real-world datasets. Handling missing values properly is crucial for building a reliable classification model.

Methods

- Remove missing values: If missing values are few and randomly distributed, you can remove those rows or columns. However, this can lead to a loss of data.
- Imputation:
 - For numerical data, you can replace missing values with the mean, median, or mode.
 - For categorical data, you can replace missing values with the most frequent category or even introduce a new “missing” category.
 - Advanced imputation: Techniques like K-Nearest Neighbors imputation or regression-based imputation can also be used for more accurate handling of missing data.

1.1.3 3. Encoding Categorical Variables

Categorical features contain discrete values like “red”, “blue”, “green” or “male”, “female”. Machine learning algorithms can’t work directly with these non-numeric values, so you need to transform them into numbers.

Methods

- **Label Encoding:** Assign a unique integer to each category. This is useful for ordinal variables (e.g., “low”, “medium”, “high”).
- **One-Hot Encoding:** Convert categorical variables into binary vectors. Each category is represented by a column, and the presence of a category is indicated by a 1, while absence is 0.
 - Example: If you have a feature “City” with values {“New York”, “San Francisco”, “Los Angeles”}, you create three binary features: City_NewYork, City_SanFrancisco, City_LosAngeles
- **Target Encoding:** Replace categorical variables with the mean of the target variable for each category. This is useful in high-cardinality categorical features (e.g., customer IDs or product IDs), especially when there are many unique categories.

Improvements:

- **Mean Encoding:** Use mean encoding or frequency encoding when there are too many unique categories. One-hot encoding might explode the feature space, but mean encoding provides a compact way to represent categorical data.
- **Categorical Embeddings:** For high-dimensional categorical features, you can use deep learning techniques like embeddings to represent categories in lower-dimensional spaces. This

is useful when the relationship between categories is more complex (e.g., word embeddings for text data).

1.1.4 4. Feature Scaling

Many classification algorithms (e.g., Logistic Regression, SVM, and KNN) perform better when numerical features are on a similar scale. Scaling ensures that no feature dominates the others because of its magnitude.

Techniques:

- **Standardization:** Transform the features to have a mean of 0 and a standard deviation of 1.
- **Normalization:** Scale the features to be in a range between [0, 1].

When to Use:

- **Logistic Regression, SVM, KNN:** These algorithms are sensitive to the scale of the data.
- **Tree-based methods (Random Forest, Decision Trees):** Not sensitive to scaling but can still benefit in terms of optimization.

1.1.5 5. Interaction Features

Sometimes, the relationship between two or more features is more important than the features themselves in isolation. Interaction terms capture these relationships and can provide valuable insights into classification tasks.

Examples

- In a healthcare dataset, the interaction between “age” and “smoking status” might be more predictive of heart disease than either feature alone.
- For predicting house prices, the interaction between “number of rooms” and “house size” may reveal patterns not captured by the individual features.

Improvements

- Use domain knowledge to create meaningful interactions. For example, in predicting customer churn, the interaction between “tenure” and “customer support calls” might be highly predictive.
- Polynomial interaction terms: Use polynomial features to automatically create interaction terms between features (e.g., $X_1 * X_2$, X_1^2 , etc.). However, this can lead to overfitting if not done carefully.

1.1.6 6. Binning Continuous Variables

Binning, or discretization, involves dividing continuous features into discrete bins or intervals. This can make relationships more visible and sometimes help models by reducing noise or handling non-linearity.

Examples

- Convert “age” into age groups (“0-18”, “19-35”, “36-60”, “60+”).

- Create bins for continuous variables such as income to separate out low, medium, and high-income groups.

Improvements

- Use domain-specific bins: For example, in finance, breaking income into meaningful bins like “low-income”, “middle-income”, and “high-income” based on the poverty threshold makes more sense than arbitrary binning.

1.1.7 7. Creating New Features Using Domain Knowledge

Sometimes, domain-specific knowledge can help you create new features that significantly boost the performance of your classification model.

Examples

- In a customer churn dataset, you could create a “loyalty” feature by calculating the total amount of time a customer has been with the company and combining it with features like “last purchase date”..
- In fraud detection, features like “average transaction amount” or “frequency of transactions” can help detect unusual behavior..

Improvements

- Combine multiple features into new ones. For example, in a car price prediction dataset, you might create a “mileage per year” feature by dividing the car’s total mileage by its age.

1.1.8 8. Dealing with Imbalanced Data

In many classification tasks, especially in fields like fraud detection or medical diagnosis, one class is significantly more common than the other. This is known as class imbalance.

Techniques

- **Resampling:** Either over-sample the minority class (e.g., using techniques like SMOTE – Synthetic Minority Over-sampling Technique) or under-sample the majority class.
- **Class weights:** Some algorithms (e.g., Logistic Regression, SVM, Random Forest) allow you to assign different weights to the classes to account for imbalance.
- **Anomaly detection:** If the minority class is extremely rare, you may treat it as an anomaly detection problem rather than a classification task.

Improvements

- Use **domain knowledge** to handle imbalance carefully. In a medical dataset, you don’t want to under-sample the majority class and lose valuable information, so focusing on techniques like SMOTE or class weights is better.

1.1.9 9. Text Data in Classification

Text data requires special feature engineering to convert raw text into features that a machine learning model can use.

Techniques

- **Bag of Words:** Convert text into a vector of word counts or word frequencies.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** This captures the importance of a word in a document relative to the corpus of documents.
- **Word Embeddings:** Advanced techniques like Word2Vec or BERT can capture semantic meanings and relationships between words.

Improvements

- **N-grams** For more context, you can include N-grams, which are sequences of N words (e.g., “New York” instead of just “New”).
- **Stemming/Lemmatization:** Reduce words to their base forms (e.g., “running” → “run”) to improve classification performance.

1.1.10 10. Dimensionality Reduction

High-dimensional data can lead to overfitting and computational inefficiency. Dimensionality reduction techniques help reduce the feature space while keeping the most important information.

Techniques:

- **PCA (Principal Component Analysis):** This is useful for reducing the number of features in your dataset while retaining the variance.
- **Feature Importance from Models:** Use tree-based models like Random Forest or XGBoost to extract feature importance scores, then remove features with low importance.

Improvements

- Be mindful of the interpretability of your model after applying dimensionality reduction. While methods like PCA are great for reducing dimensions, they can make it harder to interpret which features are actually important.

1.2 Feature Engineering in Action: Improvements for Classification Models

1.2.1 1. Automated Feature Engineering:

- Tools like FeatureTools can automate feature engineering by using Deep Feature Synthesis to generate multiple features from existing ones.
- AutoML tools like H2O.ai, DataRobot, and TPOT also automate the process of creating new features from raw data.

1.2.2 2. Feature Selection:

- Use techniques like L1 regularization (Lasso Regression) to perform feature selection by shrinking irrelevant features to zero.
- Perform recursive feature elimination (RFE) to iteratively remove the least important features.

1.2.3 3. Feature Engineering Pipelines

- Use tools like scikit-learn’s Pipeline and FeatureUnion to automate the process of transforming features. This ensures that all your transformations are reproducible and apply the same steps to training and test data.

1.3 Feature Engineering in Regression

In regression problems (e.g., predicting house prices), the goal is to create features that help predict a continuous output. Here are some tips:

Feature engineering for regression is similar in principle to feature engineering for classification, but it focuses on predicting continuous outcomes rather than discrete categories. The quality of features can greatly affect a model’s ability to accurately predict values, so careful feature engineering can lead to substantial improvements in regression performance.

1.3.1 Why is Feature Engineering Crucial in Regression?

In regression problems, the goal is to predict a continuous value, such as house prices, sales revenue, or customer satisfaction scores. The ability of the model to make accurate predictions depends heavily on how well the input features represent the underlying patterns in the data. Well-engineered features help the model learn these patterns and generalize well to unseen data.

1.3.2 Key Steps in Feature Engineering for Regression

1.3.3 1. Understanding the Domain and Data

The first step is to deeply understand the domain and the problem you are trying to solve. Domain knowledge can help identify which features may have a strong influence on the target variable and how these features should be engineered.

Example: In a house price prediction model, understanding that features like “location”, “number of bedrooms”, and “square footage” impact house prices allows you to focus on creating relevant features from these variables.

1.3.4 2. Handling Missing Values

Just like in classification, missing values are common in regression datasets and need to be handled carefully to avoid biasing the model or reducing its accuracy.

Methods:

- **Mean/Median Imputation:** Replace missing values with the mean or median of the feature.
- **Interpolation:** For time-series regression, you can use interpolation to estimate missing values..
- **Predictive Imputation:** Build a model to predict missing values based on other features.

Example: In a time-series dataset predicting stock prices, missing values could be filled in using interpolation based on previous and future stock prices.

1.3.5 3. Encoding Categorical Variables

Categorical features must be converted into numeric representations for regression models to use them.

Techniques:

- **One-Hot Encoding:** Converts categorical variables into a set of binary variables. This is useful for non-ordinal categories (e.g., car brands).
- **Label Encoding:** Assigns each category an integer value, useful for ordinal variables (e.g., “low”, “medium”, “high”).
- **Target Encoding:** Replace categories with the mean of the target variable for each category. For example, in a house price prediction problem, you could replace “neighborhood” with the average house price in that neighborhood.

Improvements:

Group Rare Categories: In regression problems with high cardinality categorical variables, group infrequent categories into a single category (e.g., “Other”) to avoid overfitting.

1.3.6 4. Feature Scaling

Regression models often benefit from features being on the same scale, especially for distance-based algorithms (like KNN) or gradient-based models (like Gradient Boosting).

Techniques:

- **Standardization:** Transform the features to have zero mean and unit variance.
- **Min-Max Scaling:** Scale the features to a $[0, 1]$ range, especially useful for models like Neural Networks.

When to Use:

- **Linear Models (Linear Regression, Ridge, Lasso):** These models often work better when the features are standardized.
- **Tree-Based Models:** While models like Decision Trees, Random Forest, and XGBoost don’t require scaling, scaling may still be useful for gradient boosting optimizations.

1.3.7 5. Creating Polynomial and Interaction Features

Linear regression assumes a linear relationship between features and the target variable. Often, real-world data contains non-linear relationships. You can improve the model’s performance by introducing polynomial and interaction features.

Techniques:

- **Polynomial Features:** Create new features by raising existing features to a power (e.g., X^2 , X^3).
- **Interaction Terms:** Multiply two or more features to capture interactions between them (e.g., $X_1 * X_2$).

Examples:

- In predicting housing prices, adding square_footage^2 might capture the non-linear effect of house size on price.
- The interaction between “number of bathrooms” and “number of bedrooms” might improve predictions, as homes with balanced ratios of rooms could be valued differently.

Improvements:

- Use **PolynomialFeatures** from scikit-learn to automatically generate polynomial and interaction terms. Be cautious, as adding too many terms can lead to overfitting.
- **Domain-Specific Interactions:** Apply domain knowledge to create meaningful interaction terms. For instance, in a car price regression model, the interaction between “mileage” and “car age” can capture more nuanced relationships.

1.3.8 6. Feature Binning (Discretization)

Binning continuous features into discrete intervals can help models capture non-linear patterns and reduce noise. This is particularly useful when the relationship between the feature and the target is non-linear.

Techniques:

- **Equal-Width Binning:** Divide the range of a continuous feature into equal-sized intervals.
- **Equal-Frequency Binning:** Divide the range so that each bin has an equal number of observations.
- **Domain-Specific Binning:** Use domain knowledge to define meaningful intervals.

Example: In predicting employee salaries, “years of experience” can be binned into categories like “0-5 years”, “6-10 years”, and “10+ years” to capture salary jumps across experience levels.

Improvements:

- Binning can reduce the impact of outliers and make models more interpretable, but too many bins can lead to information loss. Use cross-validation to tune the number of bins.

1.3.9 7. Logarithmic and Power Transformations

Sometimes, features or the target variable have skewed distributions. Skewed distributions can reduce the performance of models, especially linear regression models, which assume a normally distributed target variable.

Techniques:

- **Log Transformation:** Apply the natural logarithm to reduce the impact of large outliers (useful for highly skewed data).
- **Square Root/Exponential Transformations:** Depending on the nature of the skewness, these can normalize the distribution.

Example: In a sales prediction model, the sales amount is often skewed, with a few very high sales. Applying a log transformation to the sales feature can reduce the impact of extreme outliers.

Improvements:

- After transforming features, check whether the relationship between the features and the target has become more linear. This can improve model performance significantly.

1.3.10 8. Date and Time Features

When your dataset includes time-based data (like a sales dataset with a “date” column), you can extract valuable information from this field by creating new features.

Techniques:

- **Extract Temporal Features:** Create new features like “day of the week”, “month”, “hour of day”, or “season”.
- **Time Difference:** Create features that measure the difference between time events (e.g., “days since last purchase”).

Example: In predicting sales revenue, the “day of the week” might be crucial since sales tend to be higher on weekends.

Improvements:

You can also extract cyclical features for time data (e.g., treating “month” or “day of the week” as cyclical variables, since January is closer to December in the cycle).

1.3.11 9. Target Transformation

In regression, if the target variable is skewed or has an unusual distribution, transforming the target itself can help the model learn better.

Techniques:

- **Log Transformation:** Apply the log transformation to the target to handle positive skewness.
- **Box-Cox or Yeo-Johnson Transformations:** These transformations are more flexible than log and can handle both positive and negative values.

Improvements:

- After transforming the target, make sure to inverse transform the predictions back to the original scale to interpret results properly.

1.3.12 10. Outlier Detection and Removal

Outliers can distort the regression model’s predictions, especially in linear models. It’s important to detect and handle outliers appropriately.

Techniques:

- **Z-Score:** Calculate the Z-Score for each feature and remove rows where the Z-Score is beyond a certain threshold.
- **IQR (Interquartile Range):** Remove observations that fall outside 1.5 times the IQR.

Example: In predicting house prices, extremely low or high prices that don’t fit the market’s overall trend could be outliers that need to be removed or treated.

Improvements:

- **Outlier Treatment:** Instead of removing outliers, consider capping them (i.e., replacing extreme values with a fixed maximum or minimum value).

1.3.13 11. Dimensionality Reduction for Regression

If you have a large number of features, dimensionality reduction can simplify the model and prevent overfitting.

Techniques:

- **Principal Component Analysis (PCA):** Use PCA to reduce the number of features by extracting the most important components that explain the variance in the data.
- **Feature Selection:** Use techniques like Lasso Regression (L1 regularization) to automatically select the most important features.

Example: If your dataset has many correlated variables (e.g., house size, lot size, number of rooms), PCA can reduce redundancy and improve model performance.

Improvements:

After reducing dimensions, test the model's performance to ensure that the reduction did not cause a loss of valuable information.

1.4 Feature Engineering in Action: Improvements for Regression Models

1. Automated Feature Engineering:

- Use tools like FeatureTools or DataRobot to automate the creation of new features, especially for complex datasets.

2. Feature Selection:

- Use techniques like Ridge Regression (L2 regularization) to penalize features with low importance, which improves the model's generalization.

3. Handling Non-Linear Relationships:

- If a simple linear regression model is underperforming, try using polynomial features or logarithmic transformations to capture non-linear patterns.

1.5 Feature Engineering in Clustering

Feature engineering is crucial for clustering tasks, as it directly impacts how effectively the clustering algorithm groups similar data points. Since clustering is an unsupervised learning technique, there are no labels to guide the algorithm. Therefore, the quality and structure of the input features are what the algorithm relies on to form meaningful clusters.

1.6 Why Feature Engineering is Important for Clustering

- Improves clustering accuracy: Well-engineered features allow the clustering algorithm to find more accurate and meaningful groups.

- Reduces noise and irrelevant data: Proper feature selection and transformation can eliminate noisy or irrelevant data that might skew the clustering results.
- Deals with high dimensionality: High-dimensional datasets can make it harder for clustering algorithms to detect meaningful patterns, but feature engineering can reduce dimensionality and improve performance.

1.6.1 Key Feature Engineering Techniques for Clustering

1.6.2 1. Scaling and Normalization

- Why: Many clustering algorithms (e.g., K-Means, Hierarchical Clustering) rely on distance measures like Euclidean distance. These distance measures are highly sensitive to the scale of the features. For example, features with larger ranges will dominate the distance calculations.

Techniques:

- **Standardization:** Transform the features so that they have a mean of 0 and a standard deviation of 1.
- **Min-Max Scaling:** Scale features to a fixed range (usually $[0, 1]$).

Example: If you are clustering customers based on “age” and “income,” age could range from 18 to 80, while income might range from \$20,000 to \$200,000. Standardizing or scaling these features ensures they have equal influence on the clustering process.

1.6.3 2. Handling Categorical Variables

Clustering algorithms require numeric input, so categorical variables must be transformed into numerical representations.

Techniques:

- **One-Hot Encoding:** Convert categorical variables into binary vectors.
- **Frequency Encoding:** Replace each category with its frequency in the dataset.
- **Target Encoding:** If you have some labels or domain knowledge, you can encode categories based on the mean of a target variable (though this is semi-supervised).

Example: For customer segmentation, categorical features like “gender” or “country” can be one-hot encoded to allow the algorithm to treat each category separately.

1.6.4 3. Dimensionality Reduction

High-dimensional data can confuse clustering algorithms, making it difficult to find meaningful clusters (due to the curse of dimensionality). Reducing dimensions makes the data easier to work with.

Techniques:

- **Principal Component Analysis (PCA):** Project high-dimensional data into a lower-dimensional space while retaining as much variability as possible.

- **t-SNE (t-distributed Stochastic Neighbor Embedding):** Useful for visualizing clusters in 2D or 3D. Autoencoders: Use deep learning to compress data into a lower-dimensional representation.

Example: If you're clustering images based on pixel values, PCA can reduce thousands of pixel features to a few principal components, helping the algorithm find meaningful image clusters.

1.6.5 4. Feature Generation

New features can help expose patterns that were not apparent in the original data. This is especially useful if the original features do not clearly separate groups.

Techniques:

- **Polynomial Features:** Create interaction terms and polynomial features to capture non-linear relationships.
- **Domain-Specific Features:** Use domain knowledge to create features that are important for clustering (e.g., “average purchase frequency” for retail data).
- **Interaction Features:** Generate features by combining two or more existing features (e.g., the ratio of two features).

Example: In a retail dataset, you might create new features like “purchase frequency” or “average order value” to better group customers based on behavior.

1.6.6 5. Handling Outliers

Outliers can significantly distort clustering, especially with distance-based algorithms like K-Means.

Techniques

- **Outlier Detection:** Use Z-scores, boxplots, or isolation forest to detect and remove or treat outliers.
- **Clustering Algorithms that Handle Outliers:** Use algorithms like DBSCAN, which treats outliers as noise by design.

Example: In a dataset of customer purchases, an unusually high spender might distort the clustering of regular customers. Removing or transforming this outlier ensures that it doesn't dominate the clustering process.

1.6.7 6. Handling Correlated Features

Highly correlated features can lead to redundant information, which could skew the clustering results.

Techniques

- **Correlation Matrix:** Identify highly correlated features and drop one or more to reduce redundancy.
- **PCA:** PCA combines correlated features into new, uncorrelated principal components.

Example: In a dataset with “total assets” and “net worth” (which are likely highly correlated), removing one of these features or combining them using PCA helps reduce the dimensionality without losing information.

1.7 Feature Engineering in Action: Improvements for Clustering Models

1. Automated Feature Engineering:

- Tools like FeatureTools can be used to automate the generation of new features. This is particularly helpful when working with large or complex datasets.
- AutoML platforms such as H2O.ai can also assist in automatically creating new features and testing their impact on the clustering process.

2. Feature Selection:

- Use techniques like SelectKBest to select the most relevant features before clustering.
- Reduce dimensionality with PCA or other dimensionality reduction techniques to simplify the dataset while retaining its important properties.

3. Exploring Distance Metrics:

While Euclidean distance is the most common metric used in clustering, other distance metrics like Manhattan distance or cosine similarity can be more appropriate depending on the dataset’s characteristics. For example, when clustering text data, cosine similarity is often a better distance metric than Euclidean distance.

4. Validation and Cluster Evaluation:

Use evaluation metrics such as the silhouette score or Davies–Bouldin index to assess the quality of the clusters. Iteratively refine your feature set based on these scores to improve clustering performance.

1.7.1 Expert Tips for Data Modeling

1. Start Simple:

- Begin with simple models like Logistic Regression (for classification) or Linear Regression (for regression). These models are easier to interpret, and feature engineering has a more obvious effect.

2. Iterate on Features:

- Build your models in stages. After your first model, evaluate which features contribute the most to accuracy. Continuously refine and add more features as necessary.

3. Use Cross-Validation:

- Always use cross-validation to validate your model performance. This ensures that your model generalizes well to new data and is not overfitting to the training set.

4. Domain Knowledge is Key:

- Use knowledge of the problem domain to guide feature creation. The best features often come from a deep understanding of the data and the problem you’re trying to solve.

5. Feature Importance:

- For models like Random Forest or XGBoost, you can easily access feature importance scores to see which features are most valuable to the model's prediction. Use this information to guide your feature selection.

1.8 Feature Engineering Selection Technique

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

%matplotlib inline

plt.rcParams["figure.figsize"] = (12,9)
```

```
[2]: wine_data = load_wine()

wine_df = pd.DataFrame(
    data=wine_data.data,
    columns=wine_data.feature_names
)

wine_df['target'] = wine_data.target

wine_df.head()
```

```
[2]:
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06		0.28	2.29	5.64	1.04
1	2.76		0.26	1.28	4.38	1.05
2	3.24		0.30	2.81	5.68	1.03
3	3.49		0.24	2.18	7.80	0.86
4	2.69		0.39	1.82	4.32	1.04

	od280/od315_of_diluted_wines	proline	target	
0		3.92	1065.0	0
1		3.40	1050.0	0
2		3.17	1185.0	0
3		3.45	1480.0	0

4 2.93 735.0 0

```
[3]: from seaborn import swarmplot

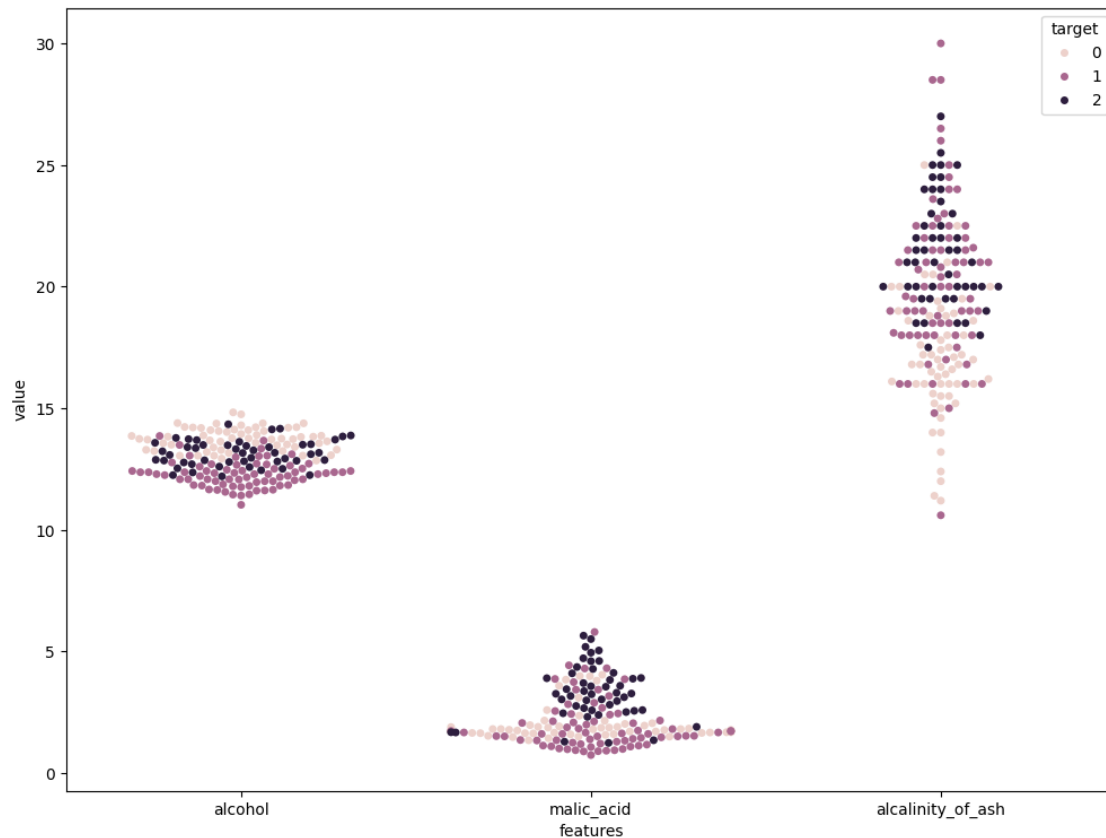
data_to_plot = pd.melt(wine_df[['alcohol', 'malic_acid', 'alcalinity_of_ash',
    ↪ 'target']],
                        id_vars='target',
                        var_name='features',
                        value_name='value')

print(data_to_plot)
swarmplot(data=data_to_plot, x='features', y='value', hue='target');
```

	target	features	value
0	0	alcohol	14.23
1	0	alcohol	13.20
2	0	alcohol	13.16
3	0	alcohol	14.37
4	0	alcohol	13.24
..	...	...	...
529	2	alcalinity_of_ash	20.50
530	2	alcalinity_of_ash	23.00
531	2	alcalinity_of_ash	20.00
532	2	alcalinity_of_ash	20.00
533	2	alcalinity_of_ash	24.50

[534 rows x 3 columns]

```
c:\Users\ikiga\AppData\Local\Programs\Python\Python311\Lib\site-
packages\seaborn\categorical.py:3399: UserWarning: 8.4% of the points cannot be
placed; you may want to decrease the size of the markers or use stripplot.
warnings.warn(msg, UserWarning)
```



```
[4]: wine_df['target'].value_counts()
```

```
[4]: target
1    71
0    59
2    48
Name: count, dtype: int64
```

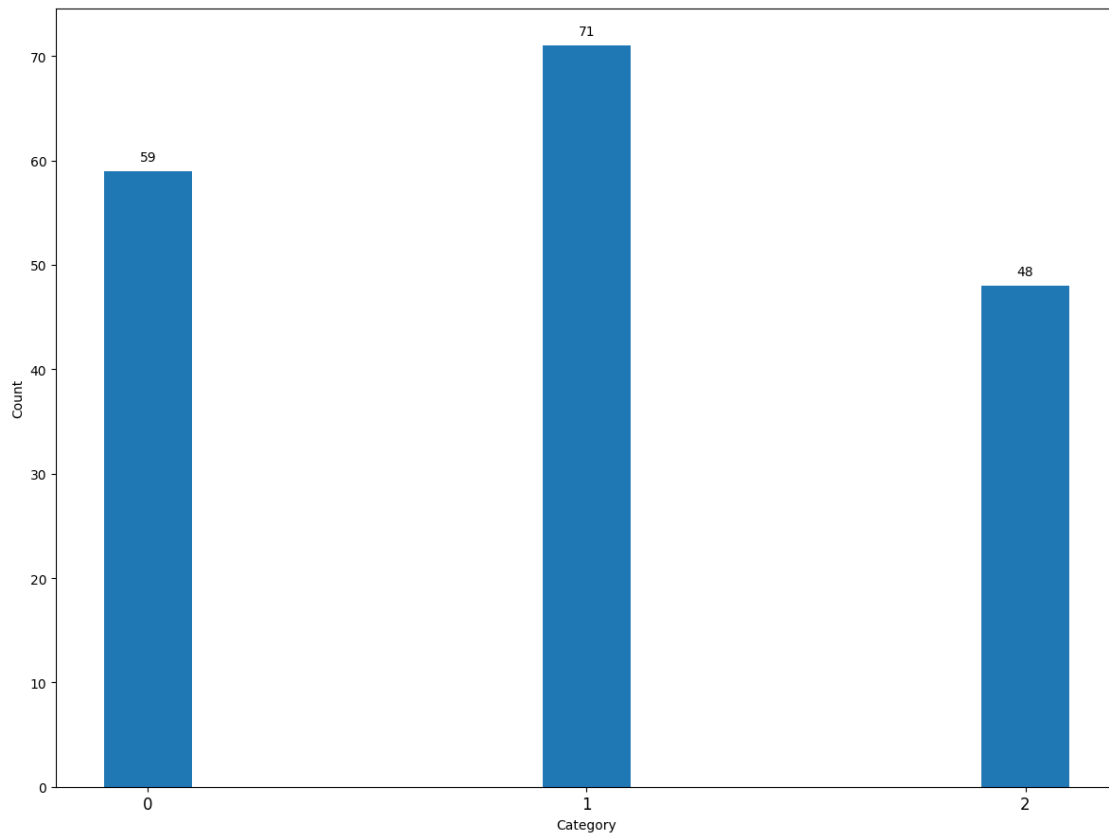
```
[5]: fig, ax = plt.subplots()

x = [0, 1, 2]
y = [59, 71, 48]

ax.bar(x, y, width=0.2)
ax.set_xlabel('Category')
ax.set_ylabel('Count')
ax.set_xticks([0,1,2])
ax.set_xticklabels([0,1,2], fontsize=12)

for index, value in enumerate(y):
```

```
plt.text(x=int(index), y=value+1, s=str(value), ha='center')
plt.tight_layout()
```



1.9 Train Test / Split

```
[6]: from sklearn.model_selection import train_test_split

X = wine_df.drop(['target'], axis=1)
y = wine_df['target']

X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.3,
                                                    shuffle=True,
                                                    stratify=y,
                                                    random_state=42)

print(X_train.shape)
print(X_test.shape)
```

(124, 13)
(54, 13)

1.10 Baseline Model: Gradient boosting Classifier with all features

```
[7]: from sklearn.ensemble import GradientBoostingClassifier
      from sklearn.metrics import f1_score

      # Initialize classifier
      gbc = GradientBoostingClassifier(max_depth=5, random_state=42)

      # Train classifier using all features
      gbc.fit(X_train, y_train)

      # Make predictions
      preds = gbc.predict(X_test)

      # Evaluate the model using the F1-score
      f1_score_all = round(f1_score(y_test, preds, average='weighted'), 3)

      print(f1_score_all)
```

0.908

1.10.1 Feature selection techniques (Variance threshold)

```
[8]: X_train_v1, X_test_v1, y_train_v1, y_test_v1 = X_train.copy(), X_test.copy(),  
      ↪ y_train.copy(), y_test.copy()  
      X_train_v1.var(axis=0)
```

```
[8]: alcohol                0.658341
      malic_acid             1.123507
      ash                    0.072433
      alcalinity_of_ash      11.471279
      magnesium              232.071532
      total_phenols          0.393226
      flavanoids             0.912299
      nonflavanoid_phenols   0.013873
      proanthocyanins        0.335108
      color_intensity        5.669722
      hue                    0.052891
      od280/od315_of_diluted_wines 0.470021
      proline                94906.710923
      dtype: float64
```

```
[9]: from sklearn.preprocessing import MinMaxScaler

      scaler = MinMaxScaler()
```

```
scaled_X_train_v1 = scaler.fit_transform(X_train_v1)
scaled_X_train_v1
```

```
[9]: array([[0.71578947, 0.19565217, 0.56451613, ..., 0.54471545, 0.61509434,
           0.84236055],
          [0.71052632, 0.15019763, 0.72043011, ..., 0.57723577, 0.54339623,
           0.81406629],
          [0.59210526, 0.17786561, 0.79569892, ..., 0.49593496, 0.56981132,
           0.48666128],
          ...,
          [0.54736842, 0.22924901, 0.74731183, ..., 0.10569106, 0.02264151,
           0.11883589],
          [0.32105263, 0.78656126, 0.6344086 , ..., 0.2195122 , 0.
           0.35731609],
          [0.42368421, 0.12252964, 0.35483871, ..., 0.40650407, 0.12075472,
           0.13904608]])
```

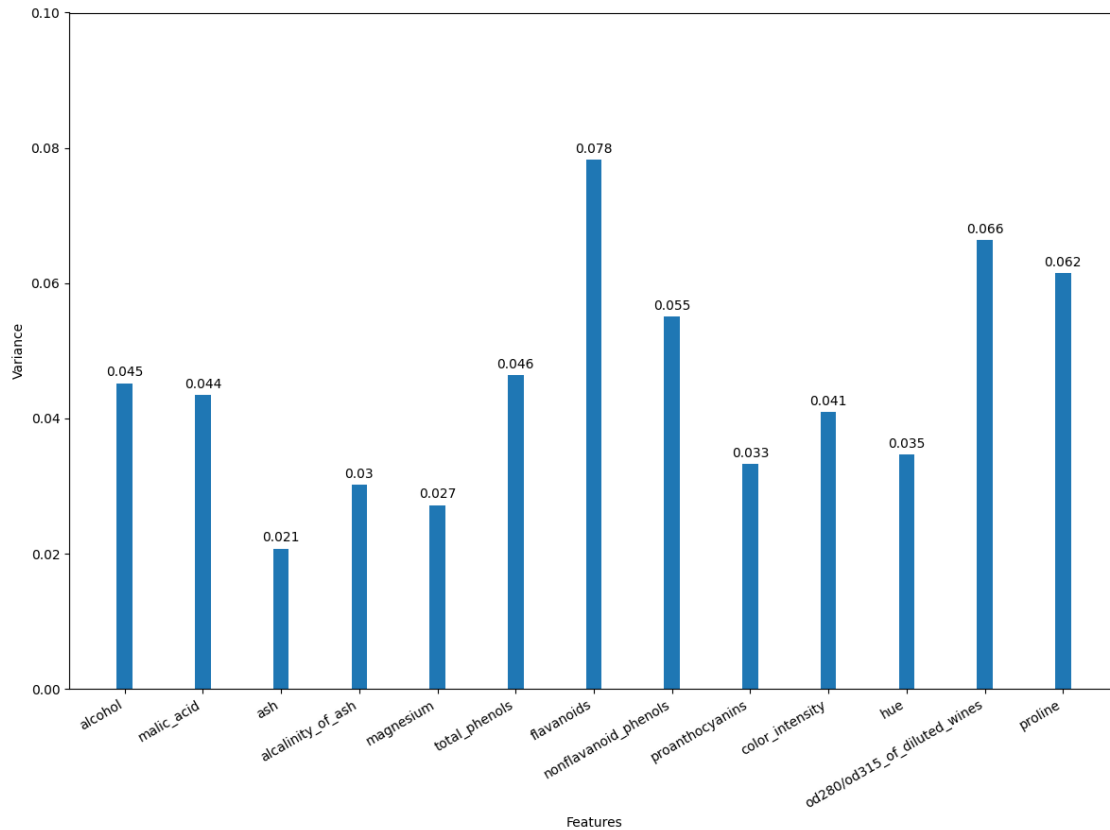
```
[10]: fig, ax = plt.subplots()

x = X.columns
y = scaled_X_train_v1.var(axis=0)

ax.bar(x, y, width=0.2)
ax.set_xlabel('Features')
ax.set_ylabel('Variance')
ax.set_ylim(0, 0.1)

for index, value in enumerate(y):
    plt.text(x=index, y=value+0.001, s=str(round(value, 3)), ha='center')

fig.autofmt_xdate()
plt.tight_layout()
```



```
[11]: sel_X_train_v1 = X_train_v1.drop(['ash', 'magnesium'], axis=1)
      sel_X_test_v1 = X_test_v1.drop(['ash', 'magnesium'], axis=1)

      gbc.fit(sel_X_train_v1, y_train)

      var_preds = gbc.predict(sel_X_test_v1)

      f1_score_var = round(f1_score(y_test_v1, var_preds, average='weighted'), 3)

      print(f1_score_var)
```

0.963

```
[12]: fig, ax = plt.subplots()

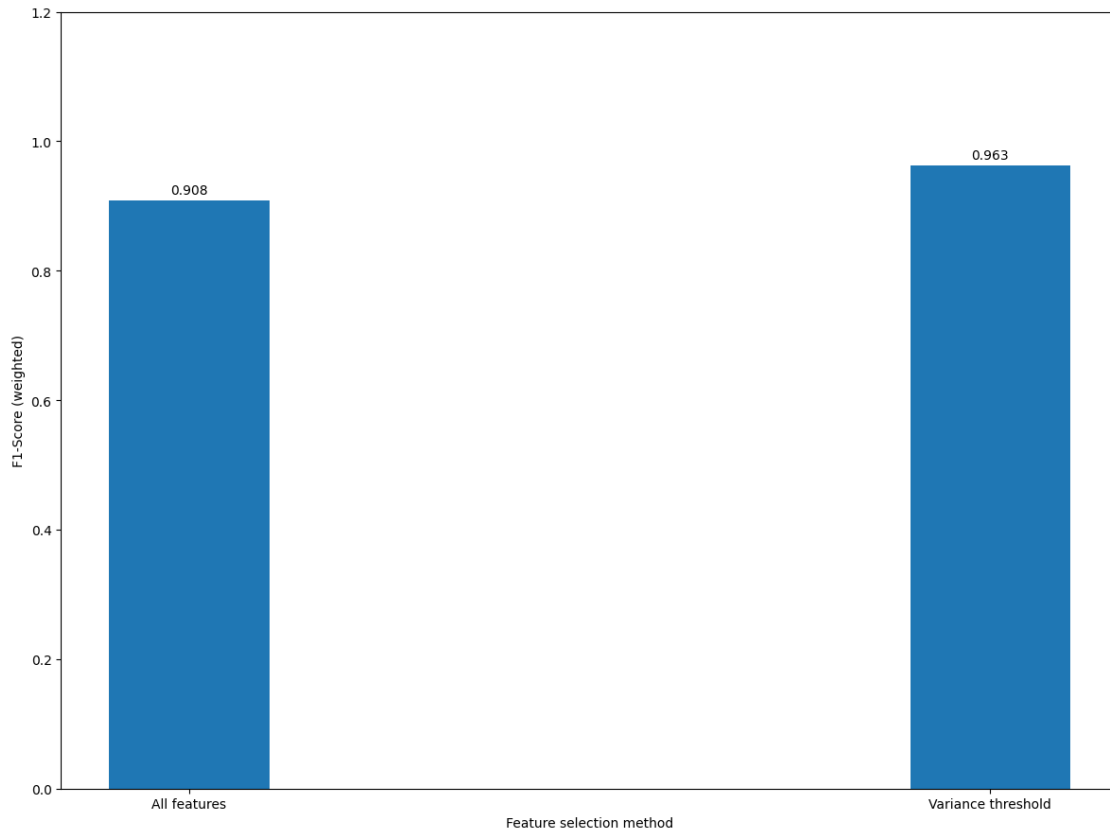
      x = ['All features', 'Variance threshold']
      y = [f1_score_all, f1_score_var]

      ax.bar(x, y, width=0.2)
      ax.set_xlabel('Feature selection method')
      ax.set_ylabel('F1-Score (weighted)')
```

```
ax.set_ylim(0, 1.2)

for index, value in enumerate(y):
    plt.text(x=index, y=value+0.01, s=str(round(value,3)), ha='center')

plt.tight_layout()
```



1.10.2 K-Best Features

```
[13]: X_train_v2, X_test_v2, y_train_v2, y_test_v2 = X_train.copy(), X_test.copy(),
↳ y_train.copy(), y_test.copy()
```

```
[14]: from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_classif

f1_score_list = []

for k in range(1, 14):
    selector = SelectKBest(mutual_info_classif, k=k)
    selector.fit(X_train_v2, y_train_v2)
```

```

sel_X_train_v2 = selector.transform(X_train_v2)
sel_X_test_v2 = selector.transform(X_test_v2)

gbc.fit(sel_X_train_v2, y_train_v2)
kbest_preds = gbc.predict(sel_X_test_v2)

f1_score_kbest = round(f1_score(y_test_v2, kbest_preds,
↪average='weighted'), 3)

f1_score_list.append(f1_score_kbest)

fig, ax = plt.subplots()

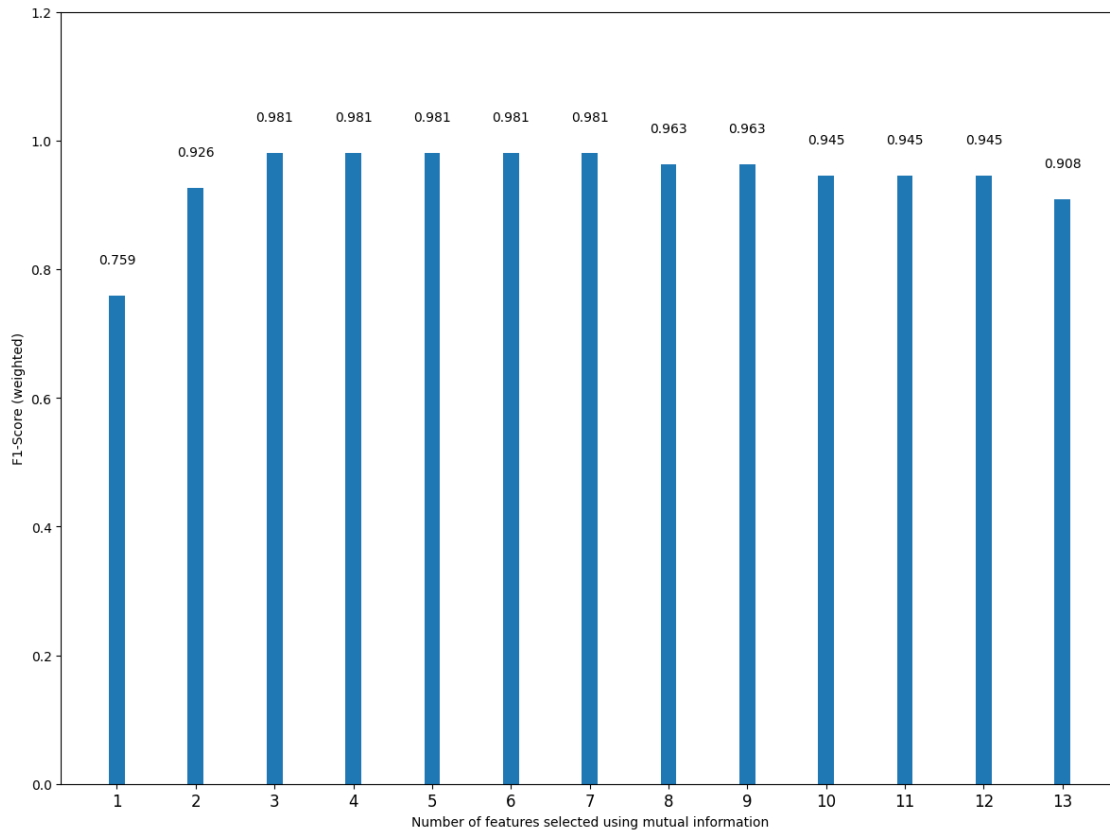
x = np.arange(1, 14)
y = f1_score_list

ax.bar(x, y, width=0.2)
ax.set_xlabel('Number of features selected using mutual information')
ax.set_ylabel('F1-Score (weighted)')
ax.set_ylim(0, 1.2)
ax.set_xticks(np.arange(1, 14))
ax.set_xticklabels(np.arange(1, 14), fontsize=12)

for i, v in enumerate(y):
    plt.text(x=i+1, y=v+0.05, s=str(v), ha='center')

plt.tight_layout()

```



```
[15]: selector = SelectKBest(mutual_info_classif, k=3)
      selector.fit(X_train_v2, y_train_v2)

      selected_feature_mask = selector.get_support()

      selected_features = X_train_v2.columns[selected_feature_mask]

      selected_features
```

```
[15]: Index(['flavanoids', 'color_intensity', 'proline'], dtype='object')
```

1.10.3 Recursive feature elimination (RFE)

```
[16]: X_train_v3, X_test_v3, y_train_v3, y_test_v3 = X_train.copy(), X_test.copy(),
      ↪ y_train.copy(), y_test.copy()
```

```
[17]: from sklearn.feature_selection import RFE

      rfe_f1_score_list = []

      for k in range(1, 14):
```

```

RFE_selector = RFE(estimator=gbc, n_features_to_select=k, step=1)
RFE_selector.fit(X_train_v3, y_train_v3)

sel_X_train_v3 = RFE_selector.transform(X_train_v3)
sel_X_test_v3 = RFE_selector.transform(X_test_v3)

gbc.fit(sel_X_train_v3, y_train_v3)
RFE_preds = gbc.predict(sel_X_test_v3)

f1_score_rfe = round(f1_score(y_test_v3, RFE_preds, average='weighted'), 3)

rfe_f1_score_list.append(f1_score_rfe)

fig, ax = plt.subplots()

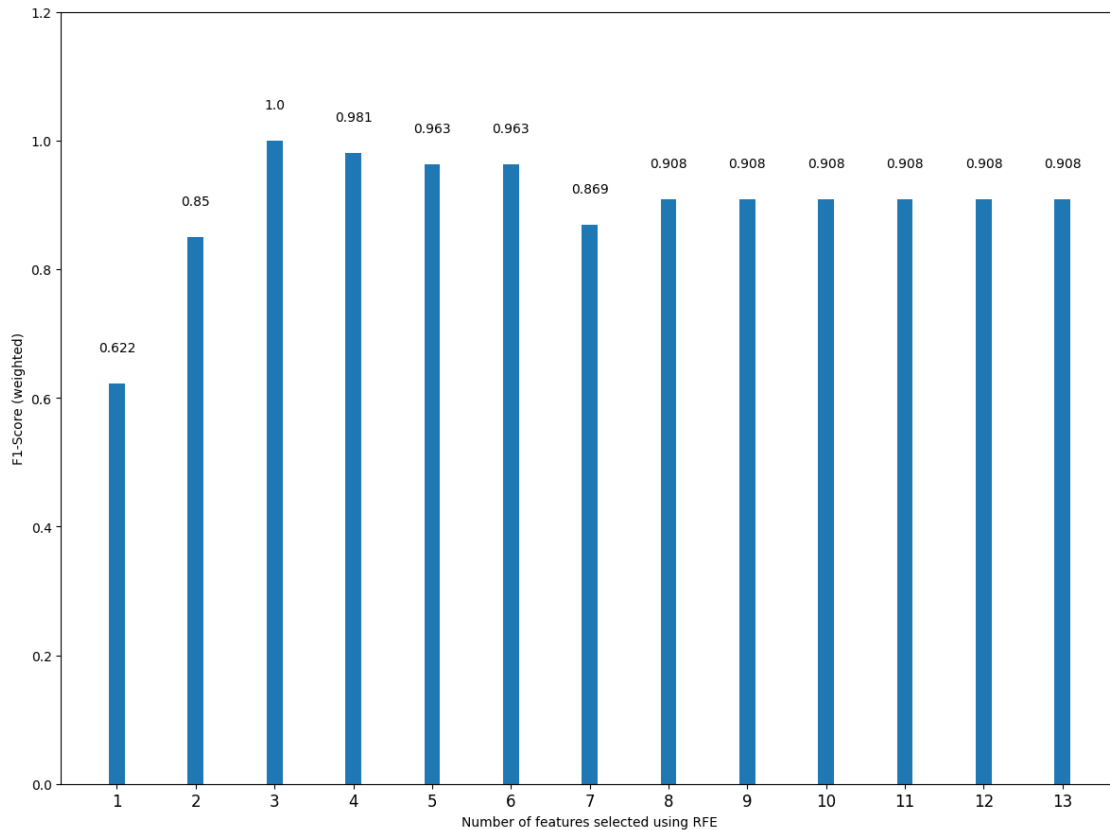
x = np.arange(1, 14)
y = rfe_f1_score_list

ax.bar(x, y, width=0.2)
ax.set_xlabel('Number of features selected using RFE')
ax.set_ylabel('F1-Score (weighted)')
ax.set_ylim(0, 1.2)
ax.set_xticks(np.arange(1, 14))
ax.set_xticklabels(np.arange(1, 14), fontsize=12)

for i, v in enumerate(y):
    plt.text(x=i+1, y=v+0.05, s=str(v), ha='center')

plt.tight_layout()

```



```
[18]: RFE_selector = RFE(estimator=gbc, n_features_to_select=3, step=10)
RFE_selector.fit(X_train_v3, y_train_v3)

selected_features_mask = RFE_selector.get_support()

selected_features = X_train_v3.columns[selected_features_mask]
selected_features
```

```
[18]: Index(['color_intensity', 'od280/od315_of_diluted_wines', 'proline'],
dtype='object')
```

1.10.4 Boruta

```
[19]: X_train_v4, X_test_v4, y_train_v4, y_test_v4 = X_train.copy(), X_test.copy(),
↳ y_train.copy(), y_test.copy()
```

```
[20]: from boruta import BorutaPy

boruta_selector = BorutaPy(gbc, random_state=42)

boruta_selector.fit(X_train_v4.values, y_train_v4.values.ravel())
```

```

sel_X_train_v4 = boruta_selector.transform(X_train_v4.values)
sel_X_test_v4 = boruta_selector.transform(X_test_v4.values)

gbc.fit(sel_X_train_v4, y_train_v4)

boruta_preds = gbc.predict(sel_X_test_v4)

boruta_f1_score = round(f1_score(y_test_v4, boruta_preds, average='weighted'), 3)
boruta_f1_score

```

[20]: 0.982

```

[21]: selected_features_mask = boruta_selector.support_

selected_features = X_train_v4.columns[selected_features_mask]
selected_features

```

[21]: Index(['malic_acid', 'ash', 'magnesium', 'flavanoids', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline'], dtype='object')

```

[22]: fig, ax = plt.subplots()

x = ['All features (13)', 'Variance threshold (11)', 'Filter - MI (3)', 'RFE (3)', 'Boruta (9)']
y = [f1_score_all, f1_score_var, 0.981, 1.0, boruta_f1_score]

ax.bar(x, y, width=0.2)
ax.set_xlabel('Feature selection method')
ax.set_ylabel('F1-Score (weighted)')
ax.set_ylim(0, 1.2)

for i, v in enumerate(y):
    plt.text(x=i, y=v+0.01, s=str(v), ha='center')

plt.tight_layout()

```

